ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к проекту «FlightFinder»

Веб-сервис для поиска авиабилетов

Разработчик: Вардумян Эрик Араратович

Преподаватель: Грачёв Николай Романович

Дата: 2026 г.

СОДЕРЖАНИЕ

1. Введение
2. Постановка задачи
3. Обзор технологий
4. Архитектура приложения
5. Структура проекта
6. Описание модулей
7. База данных
8. Реализованный функционал
9. REST API
10. Пользовательский интерфейс
11. Безопасность
12. Тестирование
13. Заключение
14. Список использованных источников
15.

1. ВВЕДЕНИЕ

Название проекта: FlightFinder

Тип проекта: Веб-приложение (Web-сервис) для поиска и бронирования авиабилетов

Актуальность: В современном мире поиск авиабилетов является востребованной задачей. Существует множество сервисов (Aviasales, Skyscanner и др.), но реализация подобного функционала с нуля позволяет глубже понять принципы веб-разработки, работу с базами данных, REST API и безопасностью.

Цель проекта: Разработать веб-сервис для поиска и сравнения авиабилетов с возможностью фильтрации по различным параметрам, с уникальным дизайном и модульной архитектурой.

2. ПОСТАНОВКА ЗАДАЧИ

2.1. Основные задачи

1. Реализовать поиск авиабилетов с динамической генерацией рейсов
2. Обеспечить фильтрацию и сортировку результатов поиска
3. Реализовать систему регистрации и авторизации пользователей
4. Создать личный кабинет с возможностью загрузки аватара
5. Реализовать функционал избранного и истории поиска
6. Обеспечить возможность бронирования билетов (в одну сторону и туда-обратно)
7. Реализовать REST API для автоподстановки городов и получения деталей рейсов
8. Обеспечить безопасность (хеширование паролей, CSRF-защита)
9. Создать адаптивный и интуитивно понятный интерфейс

2.2. Требования к реализации

- Язык программирования: Python 3.12
- Веб-фреймворк: Flask
- База данных: SQLite с SQLAlchemy ORM
- Модульная структура (разделение на Blueprint'ы)
- Соблюдение стандартов PEP 8
- Защита от CSRF-атак
- Хеширование паролей

3. ОБЗОР ТЕХНОЛОГИЙ

3.1. Backend технологии

Технология	Версия	Назначение
Python	3.12	Основной язык программирования
Flask	3.1.3	Веб-фреймворк
Flask-SQLAlchemy	3.1.1	ORM для работы с БД
Flask-WTF	1.2.2	Работа с формами и CSRF-защита
Flask-Login	0.6.3	Управление сессиями пользователей
Flask-RESTful	0.3.10	Создание REST API
Pillow	12.1.1	Обработка изображений (аватары)
pytz	2026.1	Работа с часовыми поясами

3.2. Frontend технологии

Технология	Назначение
Bootstrap 5	CSS-фреймворк для адаптивного дизайна
Bootstrap Icons	Иконки
Select2	Выпадающие списки с поиском
jQuery	AJAX-запросы
JavaScript	Динамическое обновление цен, модальные окна
Jinja2	Шаблонизатор HTML

3.3. Инструменты разработки

- IDE: PyCharm
- Контроль версий: Git
- Виртуальное окружение: venv
- Хостинг репозитория: GitHub

4. АРХИТЕКТУРА ПРИЛОЖЕНИЯ

4.1. Модульная структура

Для разделения функционала использованы Blueprint'ы:

Blueprint	Маршруты	Описание
main	/, /search, /results	Главная страница, поиск, результаты
auth	/login, /register, /logout	Аутентификация
bookings	/book/*, /my_bookings, /confirm_booking	Бронирования
profile	/profile, /upload_avatar, /clear_history	Личный кабинет
flights	/flight/*, /add_to_favorites, /remove_favorite/*	Рейсы и избранное

5. СТРУКТУРА ПРОЕКТА

```
FlightFinder/
│
├── main.py                         # Точка входа
├── requirements.txt                # Зависимости
├── .gitignore                      # Игнорируемые файлы
│
├── app/                            # Основной пакет
│   ├── __init__.py                 # Фабрика приложения
│   ├── models.py                   # Модели БД (7 классов)
│   ├── forms.py                    # WTForms (6 классов)
│   ├── utils.py                    # Утилиты (15+ функций)
│   │
│   ├── routes/                     # Маршруты (Blueprint'ы)
│   │   ├── __init__.py
│   │   ├── main.py
│   │   ├── auth.py
│   │   ├── bookings.py
│   │   ├── profile.py
│   │   └── flights.py
│   │
│   ├── templates/                  # HTML шаблоны (17 файлов)
│   └── static/                     # Статические файлы
│
├── uploads/                        # Загруженные аватарки
└── instance/                       # База данных
```

6. ОПИСАНИЕ МОДУЛЕЙ

6.1. app/init.py - Фабрика приложения

Содержит функцию create_app(), которая:

- Создаёт экземпляр Flask
- Настраивает конфигурацию
- Инициализирует расширения
- Регистрирует Blueprint'ы
- Регистрирует обработчики ошибок (404, 500)
- Добавляет фильтры шаблонов (конвертация времени)

6.2. app/models.py - Модели данных

Содержит 7 классов SQLAlchemy:

Модель	Описание	Основные поля
User	Пользователи	id, username, email, password_hash, avatar
Airport	Аэропорты	iata_code, city_ru, airport_name, timezone
Flight	Рейсы	airline, origin, destination, departure, arrival, base_price
Booking	Бронирования (в одну сторону)	booking_number, user_id, flight_id, total_price
RoundTripBooking	Бронирования (туда-обратно)	outbound_flight_id, return_flight_id
Favorite	Избранное	user_id, flight_id
SearchHistory	История поиска	origin, destination, departure_date

6.3. app/forms.py - Формы

Содержит 6 классов WTForms:

Форма	Назначение	Основные поля
LoginForm	Вход	email, password, remember
RegistrationForm	Регистрация	username, email, password, password2
SearchForm	Поиск	origin, destination, dates, passengers, class
BookingForm	Бронирование	adults, children, infants, travel_class
AvatarForm	Загрузка аватара	avatar
ConfirmBookingForm	Подтверждение	submit

6.4. app/utils.py - Утилиты

Содержит вспомогательные функции:

Работа с городами:

- format_city_name() - форматирование названий
- get_airport_info() - получение информации об аэропорте
- check_city_has_airport() - проверка наличия аэропорта
- get_alternative_cities() - поиск альтернативных городов

Генерация рейсов:

- generate_flights_for_date() - генерация рейсов на дату
- generate_stop_info() - генерация информации о пересадках

Расчёт цен:

- calculate_flight_price() - расчёт цены с учётом класса и багажа

Работа с аватарами:

- save_avatar() - сохранение загруженного аватара
- create_default_avatar() - создание аватара по умолчанию

Загрузка данных:

- add_airports_from_list() - добавление аэропортов (70+)
- generate_flights() - генерация тестовых рейсов (400+)

6.5. Маршруты (routes)

routes/main.py - основные маршруты:

- GET / - главная страница с формой поиска
- GET/POST /search - страница поиска
- GET /results - результаты с фильтрацией и сортировкой

routes/auth.py - аутентификация:

- GET/POST /login - вход
- GET/POST /register - регистрация
- GET /logout - выход

routes/bookings.py - бронирования:

- GET /my_bookings - список бронирований
- POST /book/<flight_id> - создание бронирования
- GET /confirm_booking - подтверждение
- POST /complete_booking - завершение
- GET /cancel_booking_confirm/<id> - отмена с CAPTCHA

routes/profile.py - личный кабинет:

- GET /profile - профиль с вкладками
- POST /upload_avatar - загрузка аватара
- GET /clear_history - очистка истории

routes/flights.py - рейсы:

- GET /flight/<flight_id> - детали рейса
- POST /add_to_favorites - добавление в избранное
- POST/GET /remove_favorite/<id> - удаление из избранного
- GET /select_return_flight - выбор обратного рейса

7. БАЗА ДАННЫХ

7.1. Тип и СУБД

- Тип: Реляционная
- СУБД: SQLite
- ORM: SQLAlchemy (Flask-SQLAlchemy)

7.2. Наполнение данными

Тип данных	Количество	Примечание
Аэропорты	70+	Все регионы России
Авиакомпании	10	Аэрофлот, S7, Уральские авиалинии и др.
Статические рейсы	400	При первом запуске
Динамические рейсы	8-15 на поиск	Генерируются на лету

8. РЕАЛИЗОВАННЫЙ ФУНКЦИОНАЛ

8.1. Поиск и генерация рейсов

Ключевая особенность: Рейсы не хранятся статически для каждой даты, а генерируются на лету при поиске.

Алгоритм генерации:

1. Получение информации об аэропортах отправления и назначения
2. Проверка существующих рейсов в БД на эту дату
3. Если рейсов нет - генерация 8-15 новых рейсов:
   - Случайное время вылета с учётом часового пояса
   - Случайная длительность (1-8 часов)
   - Случайное количество пересадок (0, 1, 2)
   - Вероятности: 55% без пересадок, 30% с 1, 15% с 2
   - Случайная базовая цена (3 000 - 80 000 руб.)
4. Сохранение рейсов в БД

8.2. Фильтрация и сортировка

Фильтры:

- По цене (диапазон от/до)
- По количеству пересадок (0, 1, 2+)
- Только рейсы с включённым багажом
- По времени вылета (утро, день, вечер, ночь)

Сортировка:

- По возрастанию/убыванию цены
- По времени вылета
- По длительности перелёта

8.3. Бронирование

Два типа бронирования:

1. One-way (в одну сторону) - модель Booking
2. Round-trip (туда-обратно) - модель RoundTripBooking

Процесс бронирования туда-обратно:
Поиск -> Выбор билета туда -> Выбор билета обратно -> Подтверждение -> Завершение -> Получение номера брони

Классы обслуживания и множители цен:

Класс	Множитель	Пример (цена 10 000 руб.)
Эконом	x1.0	10 000 руб.
Комфорт	x1.5	15 000 руб.
Бизнес	x2.5	25 000 руб.
Первый класс	x4.0	40 000 руб.

8.4. Личный кабинет

Вкладки:

1. Избранное - список сохранённых рейсов с возможностью удаления
2. История поиска - последние 10 запросов, очистка истории
3. Мои бронирования - все бронирования с краткой информацией

Функции:

- Загрузка и смена аватара (обработка через Pillow)
- Просмотр профиля (username, email, дата регистрации)
- Статистика (количество избранных, поисков, бронирований)

8.5. Отмена бронирования с CAPTCHA

Для отмены бронирования пользователь должен решить простой пример:

2 + 2 = ?

Это защищает от случайной отмены бронирования.

9. REST API

9.1. Эндпоинты

Метод	URL	Описание
GET	/api/airports?q=моск	Поиск городов
GET	/api/destinations?origin=Москва&q=пит	Поиск направлений
GET	/api/flight/123?travel_class=economy	Детали рейса
GET	/api/user_favorites	Избранное пользователя

9.2. Интеграция с фронтендом

API используется для:

- Автоподстановки городов в полях ввода (Select2)
- Загрузки деталей рейса в модальное окно
- Отображения статуса избранного (активные сердечки)

10. ПОЛЬЗОВАТЕЛЬСКИЙ ИНТЕРФЕЙС

10.1. Страницы

Страница	URL	Описание
Главная	/	Форма поиска, популярные направления, спецпредложения
Поиск	/search	Расширенная форма поиска
Результаты	/results	Список рейсов с фильтрами и сортировкой
Детали рейса	/flight/<id>	Полная информация + бронирование
Вход	/login	Форма авторизации
Регистрация	/register	Форма регистрации
Личный кабинет	/profile	Профиль + избранное + история + бронирования
Мои бронирования	/my_bookings	Список всех бронирований

10.2. Дизайн

- Цветовая схема: светлая тема с акцентами синего цвета (#4361ee)
- Шрифты: Inter, -apple-system, sans-serif
- Адаптивность: Bootstrap 5 обеспечивает корректное отображение на всех устройствах
- Анимации: плавные переходы при наведении на карточки рейсов

10.3. Шаблоны (Jinja2)

Всего создано 17 HTML шаблонов:

Шаблон	Назначение
base.html	Базовый шаблон (навигация, футер, flash-сообщения)
index.html	Главная страница
search.html	Расширенная форма поиска
results.html	Результаты поиска
flight_detail.html	Детали рейса
profile.html	Личный кабинет
my_bookings.html	Мои бронирования
login.html	Вход
register.html	Регистрация
confirm_booking.html	Подтверждение бронирования
select_return.html	Выбор обратного рейса
cancel_booking_confirm.html	Отмена (капча)
404.html, 500.html	Страницы ошибок
и другие	Вспомогательные

11. БЕЗОПАСНОСТЬ

11.1. Реализованные механизмы

Механизм	Реализация	Назначение
Хеширование паролей	werkzeug.security.generate_password_hash()	Пароли не хранятся в открытом виде
CSRF-защита	Flask-WTF + {{ form.hidden_tag() }}	Защита от подделки межсайтовых запросов
Валидация файлов	Проверка расширений (.jpg, .png, .gif)	Только изображения
Ограничение размера	MAX_CONTENT_LENGTH = 16MB	Защита от DoS-атак
Безопасные имена файлов	werkzeug.utils.secure_filename()	Санитизация имён файлов
Валидация форм	WTForms validators	Проверка данных на сервере

11.2. Защита от распространённых уязвимостей

Уязвимость	Защита
SQL-инъекции	ORM (SQLAlchemy) экранирует запросы
XSS	Jinja2 автоматически экранирует вывод
CSRF	Flask-WTF токены

12. ТЕСТИРОВАНИЕ

12.1. Сценарии тестирования

Сценарий	Ожидаемый результат
Регистрация нового пользователя	Пользователь создаётся, перенаправление на вход
Вход с правильными данными	Вход выполнен, перенаправление в профиль
Поиск рейсов Москва -> СПб	Отображаются рейсы с фильтрами
Фильтрация по цене	Показываются только рейсы в диапазоне
Добавление рейса в избранное	Сердечко становится красным
Бронирование билета	Создаётся бронирование с уникальным номером
Отмена бронирования с капчей	Бронирование отменяется
Загрузка аватара	Аватар сохраняется и отображается

12.2. Результаты тестирования

Все сценарии успешно пройдены. Приложение работает стабильно, ошибки не обнаружены.

13. ЗАКЛЮЧЕНИЕ

13.1. Результаты работы

В ходе выполнения проекта был разработан полнофункциональный веб-сервис FlightFinder, который:

1. Позволяет пользователям искать авиабилеты с динамической генерацией рейсов
2. Предоставляет удобные фильтры и сортировку результатов
3. Содержит систему регистрации и авторизации с хешированием паролей

4. Реализует личный кабинет с загрузкой аватаров
5. Поддерживает избранное и историю поиска
6. Обеспечивает бронирование билетов (в одну сторону и туда-обратно)
7. Включает REST API для автоподстановки городов
8. Имеет современный адаптивный дизайн

13.2. Итоговый объём кода

Компонент	Строк кода
Python код	1700
HTML шаблоны	2000
ОБЩИЙ ОБЪЁМ	3700+ строк

13.3. Вывод

Проект FlightFinder представляет собой полноценное веб-приложение, реализующее все требования учебной программы. В проекте использованы все изученные технологии, применены современные подходы к разработке (модульная архитектура, паттерн Factory, Blueprint'ы), обеспечена безопасность и удобство использования. Проект готов к сдаче и может быть представлен в качестве курсовой работы.

14. СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Документация Flask
   URL: https://flask.palletsprojects.com/
   Дата обращения: 2026
2. Документация SQLAlchemy
   URL: https://www.sqlalchemy.org/
   Дата обращения: 2026
3. Документация Flask-Login
   URL: https://flask-login.readthedocs.io/
   Дата обращения: 2026

4. Документация Flask-WTF

   URL: https://flask-wtf.readthedocs.io/

   Дата обращения: 2026

5. Документация Flask-RESTful

   URL: https://flask-restful.readthedocs.io/

   Дата обращения: 2026

6. Документация Bootstrap 5

   URL: https://getbootstrap.com/

   Дата обращения: 2026

7. Pillow (PIL) Documentation

   URL: https://pillow.readthedocs.io/

   Дата обращения: 2026

8. pytz Documentation

   URL: https://pythonhosted.org/pytz/

   Дата обращения: 2026

9. Материалы курса Яндекс.Лицея

   Тема: WEB-разработка на Flask

   2025-2026 учебный год

Дата составления: 18 апреля 2026 г.